Min Heap Construction

AlgoExpert · Medium · Heaps

The problem

- Array-backed min-heap with buildHeap (O(n) sift-down), siftUp, siftDown, peek, insert and remove.

Intuition

- A heap is an array tree; sift up after insert, sift down after removal.

Brute force vs optimal

- Insert one by one with siftUp → O(n log n)
- Bottom-up siftDown from the last parent → O(n)
- Most nodes are near the bottom and barely move

Approach

- 1. Build by sifting down from the last parent
- 2. Insert at the end and sift up
- 3. Remove by swapping root with last and sifting down

Worked example



0	1	2	3	4	5	6	7
30	102	23	17	18	9	44	12

Start: arbitrary array as a tree

Worked example



0	1	2	3	4	5	6	7
30	102	23	12	18	9	44	17

siftDown index 3 (value 17)

Worked example



siftDown index 2 (value 23)

Worked example



siftDown index 1 (value 102)

Worked example



siftDown index 0 (value 30)

Java solution (1/2)

```java
public class MinHeapConstruction {
    public static class MinHeap {
        public final List<Integer> heap;

        public MinHeap(List<Integer> array) { heap = buildHeap(new ArrayList<>(array)); }

        public List<Integer> buildHeap(List<Integer> a) {
            for (int i = (a.size() - 2) / 2; i >= 0; i--) siftDown(i, a.size() - 1, a);
            return a;
        }

        public void siftDown(int i, int end, List<Integer> h) {
            while (2 * i + 1 <= end) {
                int c = 2 * i + 1;
                if (c + 1 <= end && h.get(c + 1) < h.get(c)) c++;
                if (h.get(c) >= h.get(i)) return;
                java.util.Collections.swap(h, i, c);
                i = c;
            }
        }

        public void siftUp(int i, List<Integer> h) {
```

Java solution (2/2)

```
            while (i > 0 && h.get((i - 1) / 2) > h.get(i)) { java.util.Collections.swap(h, i, (i - 1) / 2); i = (i - 1) / 2; }
        }

        public int peek() { return heap.get(0); }

        public int remove() {
            java.util.Collections.swap(heap, 0, heap.size() - 1);
            int v = heap.remove(heap.size() - 1);
            siftDown(0, heap.size() - 1, heap);
            return v;
        }

        public void insert(int v) { heap.add(v); siftUp(heap.size() - 1, heap); }

        public boolean isEmpty() { return heap.isEmpty(); }
    }
}
```

Complexity

- Time: build O(n), insert/remove O(log n), peek O(1)
- Space: O(n)

Edge cases & pitfalls

- Last parent is (n – 2) / 2
- Pick the smaller child
- Remove: swap root with last, pop, siftDown

Interview tips & follow-ups

- Explain why build-heap is O(n)
- Follow-up: k-th largest, merge k sorted lists

Recap

- A heap is an array tree; sift up after insert, sift down after removal.
- Complexity: build O(n), insert/remove O(log n), peek O(1)